DSM REST API Reference

Base URL: `http://localhost:8080`

Optimization Flow

The typical call sequence is:

```
POST /get_options  →  POST /select_option  →  POST /advance_step (repeated)
```

`POST /get_options`

Runs the multi-objective optimizer for one or more remanufacturing routes and returns a Pareto front of solutions for each route. Results are cached in Redis for 30 minutes.

Request body

```
{
  "Batch": 14,
  "optimization_mode": "forward",
  "starting_point": [57.7089, 11.9746],
  "process_requirements": {
    "product_match": "IE3",
    "quantity": 5
  },
  "routes": {
    "RouteA": {
      "Cleaning":     { "prerequisites": [],           "requirements": { "axis_height"
      "Disassembly":  { "prerequisites": ["Cleaning"],  "requirements": {} },
      "Reassembly":   { "prerequisites": ["Disassembly"],"requirements": {} },
      "Certification":{ "prerequisites": ["Reassembly"], "requirements": {} }
    }
  }
}
```

Field	Type	Required	Description
`Batch`	integer	no	Batch identifier
`optimization_mode`	string	yes	`"forward"` (batch owner triggers) or `"reverse"` (customer triggers)
`starting_point`	`[lat, lon]`	yes	Geographic origin of the batch
`process_requirements.product_match`	string	yes	Efficiency class to match (e.g. `"IE3"`)
`process_requirements.quantity`	integer	yes	Number of motors in the batch
`routes`	object	yes	Named DAG routes; each key is a route name, each value maps process names to DAG nodes
`routes.<name>.<step>.prerequisites`	string[]	yes	Process names that must complete before this step
`routes.<name>.<step>.requirements`	object	no	Technical constraints used to filter capable actors

Success `200`

Field	Type	Description
`message`	string	Top-level status message
`optimizationResults.message`	string	Internal optimizer status message
`optimizationResults.results`	array	One entry per submitted route

Field	Type	Description
`results[].processID`	string (UUID)	Identifier used in subsequent `/select_option` calls; cached in Redis for 30 minutes
`results[].processSequence`	string[]	Topological execution order of all process names
`results[].dagPrereqs`	map<string, string[]>	Maps each process name to its prerequisite process names
`results[].optimizationResultsJson.Solutions`	array	Pareto-front solutions for this route
`Solutions[].SolNumber`	integer	0-based index used to select this solution
`Solutions[].Assignments`	array	Process-to-actor assignments in topological order; `origin` and `end` boundary nodes are excluded
`Assignments[].processName`	string	Process name
`Assignments[].actorID`	string	ID of the assigned technology provider
`Solutions[].TransportCost`	float	Total transport cost across all assignments
`Solutions[].ManufacturingCost`	float	Total manufacturing cost across all assignments

```
{
  "message": "Optimization completed successfully",
  "optimizationResults": {
    "message": "Optimization process initiated for multiple sequences.",
    "results": [
      {
        "processID": "3f2e1d0c-9a8b-7c6d-5e4f-3a2b1c0d9e8f",
        "processSequence": ["Cleaning", "Disassembly", "Reassembly", "Certification"],
        "dagPrereqs": {
          "Cleaning":     [],
          "Disassembly":  ["Cleaning"],
          "Reassembly":   ["Disassembly"],
          "Certification":["Reassembly"]
        },
        "optimizationResultsJson": {
          "Solutions": [
            {
              "SolNumber": 0,
              "Assignments": [
                { "processName": "Cleaning",      "actorID": "Dissasembler1" },
                { "processName": "Disassembly",   "actorID": "Dissasembler1" },
                { "processName": "Reassembly",    "actorID": "Dissasembler2" },
                { "processName": "Certification", "actorID": "Dissasembler1" }
              ],
              "TransportCost":     222.39,
              "ManufacturingCost": 402.00
            }
          ]
        }
      }
    ]
  }
}
```


Error responses

Status	Condition
`400`	Malformed JSON
`500`	Invalid `optimization_mode` , cost matrix failure, or optimizer unreachable
`503`	AAS API, optimizer, or MongoDB (forward mode) unavailable

`POST /select_option`

Selects a specific solution from a previous optimization run, builds an execution graph, and persists it to MongoDB.

Request body

```
{
  "processID": "3f2e1d0c-9a8b-7c6d-5e4f-3a2b1c0d9e8f",
  "solNumber": 0
}
```

Field	Type	Description
`processID`	string (UUID)	Returned by `POST /get_options`
`solNumber`	integer	0-based index of the chosen solution

Success `200`

Field	Type	Description
`message`	string	Confirmation message
`ProcessID`	string[]	Full process sequence for the chosen route
`sample`	object	Cached AAS shell data for the first actor in the sequence; structure varies by actor
`Choice.SolNumber`	integer	Index of the selected solution
`Choice.Assignments`	array	Process-to-actor assignments in topological order
`Choice.Assignments[].processName`	string	Process name
`Choice.Assignments[].actorID`	string	Assigned actor ID
`Choice.TransportCost`	float	Total transport cost for the chosen solution

Field	Type	Description
`Choice.ManufacturingCost`	float	Total manufacturing cost for the chosen solution
`executionGraph`	ExecutionGraph	Persisted workflow; see ExecutionGraph structure below

```
{
  "message": "User Selected their choice!",
  "ProcessID": ["Cleaning", "Disassembly", "Reassembly", "Certification"],
  "sample": {},
  "Choice": {
    "SolNumber": 0,
    "Assignments": [
      { "processName": "Cleaning",      "actorID": "Dissasembler1" },
      { "processName": "Disassembly",   "actorID": "Dissasembler1" },
      { "processName": "Reassembly",    "actorID": "Dissasembler2" },
      { "processName": "Certification", "actorID": "Dissasembler1" }
    ],
    "TransportCost":     222.39,
    "ManufacturingCost": 402.00
  },
  "executionGraph": {
    "graphId": "3f2e1d0c-9a8b-7c6d-5e4f-3a2b1c0d9e8f",
    "processSequence": ["Cleaning", "Disassembly", "Reassembly", "Certification"],
    "currentSteps":    ["Cleaning"],
    "totalTransportCost":     222.39,
    "totalManufacturingCost": 402.00,
    "nodes": {
      "Cleaning": {
        "processName": "Cleaning",
        "actorId":     "Dissasembler1",
        "cost":        50.0,
        "co2":         5.0,
        "next":        ["Disassembly"],
        "status":      "in_progress"
      },
      "Disassembly": {
        "processName": "Disassembly",
        "actorId":     "Dissasembler1",
        "cost":        60.0,
        "co2":         6.0,
        "next":        ["Reassembly"],
        "status":      "pending"
      },
      "Reassembly": {
        "processName": "Reassembly",
        "actorId":     "Dissasembler2",
        "cost":        80.0,
        "co2":         8.0,
```

```
        "next":        ["Certification"],
        "status":      "pending"
      },
      "Certification": {
        "processName": "Certification",
        "actorId":     "Dissasembler1",
        "cost":        30.0,
        "co2":         3.0,
        "next":        [],
        "status":      "pending"
      }
    }
  }
}
```


Error responses

Status	Condition
`400`	Malformed JSON
`404`	`solNumber` not found in cached results
`500`	Redis miss, TP data unavailable, or graph save failed

`POST /advance_step`

Marks the current in-progress steps as `"completed"` and activates their successors. Call repeatedly until `currentSteps` is empty (workflow finished).

Request body


```
{
  "graphId": "3f2e1d0c-9a8b-7c6d-5e4f-3a2b1c0d9e8f"
}
```


Field	Type	Description
`graphId`	string (UUID)	ID of the execution graph to advance

Success `200` — the updated `ExecutionGraph` ; see ExecutionGraph structure below.

```
{
  "graphId": "3f2e1d0c-9a8b-7c6d-5e4f-3a2b1c0d9e8f",
  "processSequence": ["Cleaning", "Disassembly", "Reassembly", "Certification"],
  "currentSteps":    ["Disassembly"],
  "totalTransportCost":     222.39,
  "totalManufacturingCost": 402.00,
  "nodes": {
    "Cleaning": {
      "processName": "Cleaning",
      "actorId":     "Dissasembler1",
      "cost":        50.0,
      "co2":         5.0,
      "next":        ["Disassembly"],
      "status":      "completed"
    },
    "Disassembly": {
      "processName": "Disassembly",
      "actorId":     "Dissasembler1",
      "cost":        60.0,
      "co2":         6.0,
      "next":        ["Reassembly"],
      "status":      "in_progress"
    },
    "Reassembly": {
      "processName": "Reassembly",
      "actorId":     "Dissasembler2",
      "cost":        80.0,
      "co2":         8.0,
      "next":        ["Certification"],
      "status":      "pending"
    },
    "Certification": {
      "processName": "Certification",
      "actorId":     "Dissasembler1",
      "cost":        30.0,
      "co2":         3.0,
      "next":        [],
      "status":      "pending"
    }
  }
}
```

When `currentSteps` is `[]`, the workflow is complete and all nodes have `status: "completed"`.

Error responses

Status	Condition
`400`	Missing or malformed `graphId`
`500`	Graph not found or DB error

`GET /execution_graphs`

Returns all saved execution graphs, sorted by `graphId`.

No parameters.

Success `200` — array of `ExecutionGraph` objects; see ExecutionGraph structure below. Returns `[]` when none exist.

```
[
  {
    "graphId": "3f2e1d0c-9a8b-7c6d-5e4f-3a2b1c0d9e8f",
    "processSequence": ["Cleaning", "Disassembly", "Reassembly", "Certification"],
    "currentSteps":    ["Cleaning"],
    "totalTransportCost":     222.39,
    "totalManufacturingCost": 402.00,
    "nodes": {
      "Cleaning": {
        "processName": "Cleaning",
        "actorId":     "Dissasembler1",
        "cost":        50.0,
        "co2":         5.0,
        "next":        ["Disassembly"],
        "status":      "in_progress"
      },
      "Disassembly": {
        "processName": "Disassembly",
        "actorId":     "Dissasembler1",
        "cost":        60.0,
        "co2":         6.0,
        "next":        ["Reassembly"],
        "status":      "pending"
      },
      "Reassembly": {
        "processName": "Reassembly",
        "actorId":     "Dissasembler2",
        "cost":        80.0,
        "co2":         8.0,
        "next":        ["Certification"],
        "status":      "pending"
      },
      "Certification": {
        "processName": "Certification",
        "actorId":     "Dissasembler1",
        "cost":        30.0,
        "co2":         3.0,
        "next":        [],
        "status":      "pending"
      }
    }
```

```
  }
]
```

`GET /get_batch_data?graphId=<id>`

Retrieves a single execution graph by its ID.

Query parameter

Parameter	Required	Description
`graphId`	yes	UUID of the execution graph

Success `200` — single `ExecutionGraph` object; see ExecutionGraph structure below.

```
{
  "graphId": "3f2e1d0c-9a8b-7c6d-5e4f-3a2b1c0d9e8f",
  "processSequence": ["Cleaning", "Disassembly", "Reassembly", "Certification"],
  "currentSteps":    ["Reassembly"],
  "totalTransportCost":     222.39,
  "totalManufacturingCost": 402.00,
  "nodes": {
    "Cleaning": {
      "processName": "Cleaning",
      "actorId":     "Dissasembler1",
      "cost":        50.0,
      "co2":         5.0,
      "next":        ["Disassembly"],
      "status":      "completed"
    },
    "Disassembly": {
      "processName": "Disassembly",
      "actorId":     "Dissasembler1",
      "cost":        60.0,
      "co2":         6.0,
      "next":        ["Reassembly"],
      "status":      "completed"
    },
    "Reassembly": {
      "processName": "Reassembly",
      "actorId":     "Dissasembler2",
      "cost":        80.0,
      "co2":         8.0,
      "next":        ["Certification"],
      "status":      "in_progress"
    },
    "Certification": {
      "processName": "Certification",
      "actorId":     "Dissasembler1",
      "cost":        30.0,
      "co2":         3.0,
      "next":        [],
      "status":      "pending"
    }
  }
}
```

Error responses

Status	Condition
`400`	`graphId` parameter missing
`404`	Graph not found
`500`	DB error

ExecutionGraph Structure

Shared by `POST /select_option`, `POST /advance_step`, `GET /execution_graphs`, and `GET /get_batch_data`.

Field	Type	Description
`graphId`	string (UUID)	Unique identifier for this workflow instance
`processSequence`	string[]	All process names in topological execution order
`currentSteps`	string[]	Process names currently executing; `[]` when the workflow is complete
`totalTransportCost`	float	Sum of transport costs across all actor assignments
`totalManufacturingCost`	float	Sum of manufacturing costs across all actor assignments
`nodes`	map<string, ProcessNode>	All process nodes, keyed by process name

ProcessNode fields

Field	Type	Description
`processName`	string	Name of the process step

Field	Type	Description
`actorId`	string	ID of the technology provider assigned to this step
`cost`	float	Manufacturing cost for this step
`co2`	float	CO_2 emissions for this step
`next`	string[]	Process names that become active after this step completes
`status`	string	`"pending"` · `"in_progress"` · `"completed"`

Purchase Proposals

`POST /new_proposal`

Creates a new customer purchase proposal.

Request body

```
{
  "proposalId":       "prop-001",
  "customerId":       "cust-123",
  "customerLocation": { "type": "Point", "coordinates": [11.9746, 57.7089] },
  "cardinality":      5,
  "productType":      "IE3",
  "requiredParameters": {
    "ratedPower":      7.5,
    "axisHeight":      132,
    "efficiencyClass": "IE3",
    "phaseN":          3
  }
}
```

`coordinates` is `[longitude, latitude]`. `phaseN` is optional.

Success `201` — the full stored Proposal object; see Proposal structure below.

```
{
  "_id":         "60c72b2f9b1d8b3a4c8e4d1a",
  "proposalId": "prop-001",
  "customerId": "cust-123",
  "customerLocation": { "type": "Point", "coordinates": [11.9746, 57.7089] },
  "cardinality": 5,
  "productType": "IE3",
  "requiredParameters": {
    "ratedPower":      7.5,
    "axisHeight":      132,
    "efficiencyClass": "IE3",
    "phaseN":          3
  },
  "createdAt": "2026-06-22T10:00:00Z",
  "updatedAt": "2026-06-22T10:00:00Z"
}
```


Error responses

Status	Condition
`400`	Missing required fields
`409`	`proposalId` already exists
`500`	DB insert failed

`GET /purchase_proposals`

Lists proposals with optional filtering and pagination.

Query parameters

Parameter	Required	Default	Description
`customerId`	no	—	Filter by customer ID
`productType`	no	—	Filter by product type (e.g. `IE3`)
`efficiencyClass`	no	—	Filter by efficiency class

Parameter	Required	Default	Description
`skip`	no	`0`	Number of records to skip
`limit`	no	`100`	Max records to return (hard cap: 1000)

Success `200` — array of Proposal objects; see Proposal structure below. Returns `[]` when no proposals match.


```
[
  {
    "_id":        "60c72b2f9b1d8b3a4c8e4d1a",
    "proposalId": "prop-001",
    "customerId": "cust-123",
    "customerLocation": { "type": "Point", "coordinates": [11.9746, 57.7089] },
    "cardinality": 5,
    "productType": "IE3",
    "requiredParameters": {
      "ratedPower":      7.5,
      "axisHeight":      132,
      "efficiencyClass": "IE3",
      "phaseN":          3
    },
    "createdAt": "2026-06-22T10:00:00Z",
    "updatedAt": "2026-06-22T10:00:00Z"
  }
]
```


Proposal Structure

Shared by `POST /new_proposal` and `GET /purchase_proposals`.

Field	Type	Description
`_id`	string (ObjectID)	MongoDB-generated document ID
`proposalId`	string	Business identifier for the proposal
`customerId`	string	Customer identifier

Field	Type	Description
`customerLocation`	GeoPoint	GeoJSON Point representing the customer's location
`customerLocation.type`	string	Always `"Point"`
`customerLocation.coordinates`	[float, float]	`[longitude, latitude]`
`cardinality`	integer	Number of motors requested (must be > 0)
`productType`	string	Motor efficiency class requested (e.g. `"IE3"`)
`requiredParameters.ratedPower`	float	Rated power in kW
`requiredParameters.axisHeight`	integer	Axis height in mm
`requiredParameters.efficiencyClass`	string	Efficiency class (e.g. `"IE3"`)
`requiredParameters.phaseN`	integer | omitted	Number of phases; omitted from response when not set
`createdAt`	string (ISO 8601)	Timestamp when the record was created
`updatedAt`	string (ISO 8601)	Timestamp of the last update

Products

`GET /products`

Returns all registered motor products, sorted by `id` ascending.

No parameters.

Success `200` — array of Product objects; see Product structure below. Returns `[]` when none exist.

```
[
  {
    "id":    "IE3-MOTOR-002",
    "cls":   "IE3",
    "axis":  100,
    "pow":   5.5,
    "batch": "Batch #26",
    "components": {
      "bearings": "Good",
      "stator":   "Good",
      "rotor":    "Degraded",
      "housing":  "Good",
      "windings": "Good",
      "shaft":    "Good"
    },
    "ownerHistory":    ["Volvo Logistics GmbH"],
    "inspectionNotes": "Rotor shows minor deformation on inner ring",
    "createdAt": "2026-06-22T10:00:00Z",
    "updatedAt": "2026-06-22T10:00:00Z"
  },
  {
    "id":             "IE4-MOTOR-004",
    "cls":            "IE4",
    "axis":           160,
    "pow":            22.0,
    "batch":          "Batch #64",
    "components":     null,
    "ownerHistory":   null,
    "inspectionNotes":null,
    "createdAt": "2026-06-22T10:00:00Z",
    "updatedAt": "2026-06-22T10:00:00Z"
  }
]
```


GET /products/:id

Returns a single product by its ID.

Success `200` — single Product object; see Product structure below.

```
{
  "id":    "IE4-MOTOR-001",
  "cls":   "IE4",
  "axis":  132,
  "pow":   11.0,
  "batch": "Batch #14",
  "components": {
    "bearings":  "Worn",
    "stator":    "Good",
    "rotor":     "Good",
    "housing":   "Good",
    "windings":  "Worn",
    "shaft":     "Good"
  },
  "ownerHistory":    ["NordicPlant AB"],
  "inspectionNotes": "Bearings show early wear signs",
  "createdAt": "2026-06-22T10:00:00Z",
  "updatedAt": "2026-06-22T10:00:00Z"
}
```


Error responses

Status	Condition
404	Product not found
500	DB error

`POST /products`

Registers a new motor product.

Request body

```
{
  "id":    "IE4-MOTOR-001",
  "cls":   "IE4",
  "axis":  132,
  "pow":   11.0,
  "batch": "Batch #14",
  "components": {
    "bearings":  "Worn",
    "stator":    "Good",
    "rotor":     "Good",
    "housing":   "Good",
    "windings":  "Worn",
    "shaft":     "Good"
  },
  "ownerHistory":    ["NordicPlant AB"],
  "inspectionNotes": "Bearings show early wear signs"
}
```

`batch`, `components`, `ownerHistory`, and `inspectionNotes` can be omitted or set to `null`. All six component fields must be present if `components` is provided.

Success `201` — the stored Product object; see Product structure below.

```
{
  "id":    "IE4-MOTOR-001",
  "cls":   "IE4",
  "axis":  132,
  "pow":   11.0,
  "batch": "Batch #14",
  "components": {
    "bearings":  "Worn",
    "stator":    "Good",
    "rotor":     "Good",
    "housing":   "Good",
    "windings":  "Worn",
    "shaft":     "Good"
  },
  "ownerHistory":    ["NordicPlant AB"],
  "inspectionNotes": "Bearings show early wear signs",
  "createdAt": "2026-06-22T10:00:00Z",
  "updatedAt": "2026-06-22T10:00:00Z"
}
```


Error responses

Status	Condition
`400`	Missing required fields (`id` , `cls` , `axis` , `pow`) or invalid component status value
`409`	Product with this `id` already exists
`500`	DB error

`GET /products/:id/history`

Returns the service history for a product. The history record is stored independently and may exist even if the product is not yet registered (or vice versa).

Success `200` — single HistoricalRecord object; see HistoricalRecord structure below.

```
{
  "owners":      ["EnergyTech SE", "NordicPlant AB"],
  "age":         7,
  "lastService": "2023-06",
  "notes":       "Bearing replacement in 2022"
}
```

Error responses

Status	Condition
`404`	No history record found for this product ID
`500`	DB error

Product Structure

Shared by `GET /products`, `GET /products/:id`, and `POST /products`.

Field	Type	Description
`id`	string	Unique motor identifier (e.g. `"IE4-MOTOR-001"`)
`cls`	string	Efficiency class (e.g. `"IE2"`, `"IE3"`, `"IE4"`)
`axis`	float	Axis height in mm
`pow`	float	Rated power in kW
`batch`	string | null	Batch identifier; `null` when not yet assigned to a batch
`components`	Components | null	Per-component condition; `null` until the first inspection
`components.bearings`	string	Bearing condition: `"Good"` · `"Worn"` · `"Degraded"` · `"Failed"`
`components.stator`	string	Stator condition (same values)
`components.rotor`	string	Rotor condition (same values)

Field	Type	Description
`components.housing`	string	Housing condition (same values)
`components.windings`	string	Windings condition (same values)
`components.shaft`	string	Shaft condition (same values)
`ownerHistory`	string[] | null	Previous owners in chronological order; `null` until first inspection
`inspectionNotes`	string | null	Free-text notes from the latest inspection; `null` until first inspection
`createdAt`	string (ISO 8601)	Timestamp when the record was created
`updatedAt`	string (ISO 8601)	Timestamp of the last update

HistoricalRecord Structure

Returned by `GET /products/:id/history`.

Field	Type	Description
`owners`	string[]	Previous owners in chronological order
`age`	integer	Age of the motor in years
`lastService`	string	Year-month of the most recent service (e.g. `"2024-03"`)
`notes`	string	Free-text service history notes